Implementation and Exploration of Link Prediction Techniques in Complex Networks

Ojus Krishna
B.Tech CSE - 24075099

Abstract

Link prediction is a fundamental problem in network science which aims to predict missing or future links in a graph. In this project, we implement and analyze multiple link prediction techniques including similarity-based, probabilistic, and learning-based models. The methods are evaluated on real-world datasets using standard metrics such as AUC, AUPR, Average Precision and Recall@K.

Contents

1 Introduction **3**

2 Methodology **3**
2.1 Data Splitting . . . 3
2.2 Evaluation Metrics . . . 3

3 Implemented Link Prediction Methods **4**
3.1 Similarity-Based Methods . . . 4
3.1.1 Local Similarity Indices . . . 4
3.1.2 Global Similarity Indices . . . 4
3.1.3 Quasi-Local Similarity Indices . . . 4
3.2 Probabilistic Models . . . 5
3.3 Learning-Based Methods . . . 5

4 Similarity-Based Methods **5**
4.1 Local Similarity Indices . . . 5
4.2 Global Similarity Indices . . . 6
4.3 Quasi-Local Methods . . . 7

5 Probabilistic Models **8**
5.1 Stochastic Block Model (SBM) . . . 8

6 Learning-Based Models **9**
6.1 Random Forest Classifier . . . 9

7 Experimental Setup **11**
7.1 Datasets . . . 11
7.2 Implementation Details . . . 11

8 Performance Metrics **12**
8.1 Area Under ROC Curve (AUC) 12
8.2 Area Under Precision-Recall Curve (AUPR) 12
8.3 Precision 13
8.4 Recall 13
8.5 Average Precision 13
8.6 Recall@k 13

9 Results **14**
9.1 AUC Comparison 14
9.2 AUPR Comparison 14
9.3 Average Precision Comparison 15
9.4 Recall@K Comparison 15

10 Comparison of Methods **16**
10.1 Local vs Global Methods 16
10.2 Community-Aware Methods 16
10.3 Probabilistic vs Heuristic Methods 16
10.4 Learning-Based Methods 16
10.5 Effect of Evaluation Metrics 17

11 Discussion **17**

12 Conclusion **18**

1 Introduction

Complex networks are widely used to represent real-world systems such as social, biological, and communication networks, where nodes denote entities and edges represent interactions. One of the key problems in network science is *link prediction*, which aims to estimate the likelihood of a connection between two nodes that are not currently linked.

Given a graph $G(V, E)$, the objective is to identify missing or potential links among all possible node pairs. This problem has important applications in areas such as social network recommendations, protein–protein interaction prediction, and recommender systems.

Various approaches have been proposed for link prediction, broadly categorized into similarity-based methods, probabilistic models, and learning-based techniques. Similarity-based methods assign scores based on structural properties of the network and can be classified into local, global, and quasi-local methods depending on the extent of network information used. Local methods are computationally efficient, while global methods capture more structural information at higher computational cost. Quasi-local methods provide a balance between these approaches.

In this project, we implement and explore multiple link prediction techniques, including local similarity indices (e.g., Common Neighbors, Adamic-Adar), global methods (e.g., Katz Index, Shortest Path), quasi-local methods, as well as probabilistic (SBM) and learning-based (Random Forest) models. The performance of these methods is evaluated using standard metrics such as AUC, AUPR, Average Precision, and Recall@K.

The goal of this work is to gain practical understanding of different link prediction techniques and analyze how network structure influences their performance.

Code Availability The implementation of all link prediction methods and experimental results is available on GitHub:

`https://github.com/Ojuice6207/Link-Prediction-Analysis`

2 Methodology

2.1 Data Splitting

- 90% edges used for training
- 10% edges used for testing

2.2 Evaluation Metrics

- AUC (Area Under ROC Curve)
- AUPR (Area Under Precision-Recall Curve)
- Average Precision
- Recall@K

3 Implemented Link Prediction Methods

In this project, link prediction techniques were implemented following the taxonomy presented in the literature.

3.1 Similarity-Based Methods

3.1.1 Local Similarity Indices

- Common Neighbors (CN)
- Jaccard Coefficient
- Adamic-Adar Index
- Preferential Attachment
- Resource Allocation Index
- Cosine Similarity
- Sorensen Index
- CAR-based Preferential Attachment
- Hub Promoted Index (HPI)
- Local Naive Bayes Common Neighbors (LNBCN)
- Clustering Coefficient Link Prediction (CCLP)

3.1.2 Global Similarity Indices

- Katz Index
- Shortest Path Index
- SimRank
- Leicht-Holme-Newman Global Index (LHNG)

3.1.3 Quasi-Local Similarity Indices

- Local Path Index
- L3 Index
- CH2-L2
- CH3-L3

3.2 Probabilistic Models

- Stochastic Block Model (SBM)

3.3 Learning-Based Methods

- Random Forest Classifier

4 Similarity-Based Methods

4.1 Local Similarity Indices

1. Common Neighbors:
$$S(x, y) = |\Gamma(x) \cap \Gamma(y)|$$
The likelihood of a link between two nodes increases with the number of common neighbors. This method performs well in real-world networks due to strong clustering properties.

2. Jaccard Coefficient:
$$S(x, y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x) \cup \Gamma(y)|}$$
This index normalizes common neighbors by total neighbors. It reduces bias toward high-degree nodes but often performs worse than CN.

3. Adamic-Adar:
$$S(x, y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{\log k_z}$$
It assigns higher weights to low-degree common neighbors. This reflects that rare connections contribute more to similarity.

4. Preferential Attachment:
$$S(x, y) = k_x \cdot k_y$$
Based on scale-free network growth, high-degree nodes are more likely to connect. It is simple but performs poorly in many real networks.

5. Resource Allocation:
$$S(x, y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{k_z}$$
Derived from resource flow dynamics, this index penalizes hubs more strongly than AA. It performs well in networks with high clustering.

6. Cosine Similarity:
$$S(x, y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{\sqrt{k_x k_y}}$$
Measures similarity using the cosine of neighborhood vectors. It considers both common neighbors and node degrees.

7. Sorensen Index:

$$S(x,y) = \frac{2|\Gamma(x) \cap \Gamma(y)|}{k_x + k_y}$$

A variant of Jaccard that emphasizes shared neighbors. It is more robust against outliers.

8. CAR Index:

$$S(x,y) = CN(x,y) \times LCL(x,y)$$

where

$$LCL(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{|\gamma(z)|}{2}$$

This index considers links among common neighbors. It improves prediction using local community structure.

9. Hub Promoted Index (HPI):

$$S(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{\min(k_x, k_y)}$$

It promotes links between low-degree nodes and hubs. Useful in hierarchical and modular networks.

10. LNBCN:

$$S(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \left[\log\left(\frac{C(z)}{1 - C(z)}\right) + \log\left(\frac{1-\rho}{\rho}\right) \right]$$

This method uses a Naive Bayes framework. It assigns different weights to common neighbors based on clustering.

11. CCLP:

$$S(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} C(z)$$

It uses clustering coefficients of common neighbors. Nodes in tightly connected regions contribute more.

4.2 Global Similarity Indices

1. Katz Index:

$$S(x,y) = \sum_{l=1}^{\infty} \beta^l (A^l)_{xy}$$

It sums over all paths between nodes with exponential damping. Shorter paths contribute more to similarity.

2. Shortest Path:

$$S(x,y) = -|d(x,y)|$$

Nodes with smaller shortest path distance are more likely to connect. It ignores multiple paths and has lower accuracy.

3. SimRank:

$$S(x,y) = \begin{cases} \frac{\alpha}{k_x k_y} \sum_{i=1}^{k_x} \sum_{j=1}^{k_y} S(\Gamma_i(x), \Gamma_j(y)) & x \neq y \\ 1 & x = y \end{cases}$$

It defines similarity recursively: nodes are similar if their neighbors are similar. It captures structural context effectively.

4. Leicht–Holme–Newman (LHNG):

$$S = \phi AS + \psi I$$

This is a recursive similarity model incorporating neighbor similarity and self-similarity. It generalizes Katz and considers global structure.

4.3 Quasi-Local Methods

1. Local Path:

$$S = A^2 + \epsilon A^3$$

It extends common neighbors by including paths of length 3. Provides a balance between accuracy and efficiency.

2. L3 Index:

$$S(x,y) = \sum_{u,v} \frac{a_{x,u} a_{u,v} a_{v,y}}{k_u k_v}$$

Uses paths of length 3 instead of 2. Effective in biological networks where triadic closure fails.

3. CH2-L2:

$$S(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1 + c_z}{1 + o_z}$$

It rewards internal local community links and penalizes external ones. Captures local community structure effectively.

4. CH3-L3:

$$S(x,y) = \sum_{z_1 \in \Gamma(x), z_2 \in \Gamma(y)} \frac{a_{z_1,z_2}}{\sqrt{(1+\tilde{c}_{z_1})(1+\tilde{c}_{z_2})}\sqrt{(1+\tilde{o}_{z_1})(1+\tilde{o}_{z_2})}}$$

Extends CH2 using paths of length 3. It captures deeper structural relationships in the network.

5 Probabilistic Models

5.1 Stochastic Block Model (SBM)

The stochastic block model (SBM) is a probabilistic generative model used to represent the structural organization of complex networks. In this framework, nodes are assigned to different groups, known as blocks or communities, and the existence of edges between nodes is governed by the group memberships. This allows SBM to model a variety of structural patterns, including community-based interactions and functional roles of nodes within the network.

Let A^o denote the observed adjacency matrix of the network. The objective is to estimate the probability that a link exists between two vertices x and y. This probability is defined as

$$R_{xy} = p_{BM}(A_{xy} = 1 \mid A^o), \tag{1}$$

which quantifies the likelihood of a connection between x and y given the observed network under the SBM formulation.

Group Assignment In SBM, every node is assigned to a specific block. For a network consisting of N nodes, the partitioning of nodes can be represented as

$$P = \{\sigma_1, \sigma_2, \ldots, \sigma_N\}, \tag{2}$$

where σ_i denotes the block label of node i.

To describe interactions between blocks, two quantities are introduced:

- $l_{\alpha\beta}$: number of observed edges between block α and block β
- $r_{\alpha\beta}$: total number of possible edges between these blocks

The value of $r_{\alpha\beta}$ is computed as

$$r_{\alpha\beta} = \begin{cases} n_\alpha n_\beta, & \alpha \neq \beta \\ \frac{n_\alpha(n_\alpha - 1)}{2}, & \alpha = \beta \end{cases} \tag{3}$$

where n_α represents the number of nodes in block α.

Computation of Link Probability The likelihood of a connection between nodes x and y is determined by considering multiple possible partitions of the network. For a specific partition, the probability of a link between the corresponding blocks is given by

$$\frac{l_{\sigma_x\sigma_y} + 1}{r_{\sigma_x\sigma_y} + 2}. \tag{4}$$

To obtain the final reliability score, this value is averaged across a set of candidate partitions:

$$R_{xy} = \frac{1}{Z} \sum_{P \in P^*} \left(\frac{l_{\sigma_x\sigma_y} + 1}{r_{\sigma_x\sigma_y} + 2} \right) \exp[-H(P)]. \tag{5}$$

Here, Z is a normalization constant, and $H(P)$ evaluates how well a given partition explains the observed network.

Partition Quality Measure Each partition is associated with a score defined as

$$H(P) = \sum_{\alpha \leq \beta} \left[\ln(r_{\alpha\beta}) + \ln \binom{r_{\alpha\beta}}{l_{\alpha\beta}} \right]. \tag{6}$$

This score represents the likelihood of observing the current edge configuration under the given partition.

Sampling of Partitions Since the total number of possible partitions increases exponentially with the number of nodes, evaluating all partitions directly is computationally infeasible. Therefore, a Metropolis Markov Chain Monte Carlo (MCMC) approach is used to sample partitions with high probability.

Starting from an initial random partition, the algorithm repeatedly selects a node and proposes assigning it to a different block. The change in the partition score is computed as

$$\Delta H = H(P_{new}) - H(P_{old}). \tag{7}$$

The proposed change is accepted with probability

$$\min(1, e^{-\Delta H}), \tag{8}$$

which biases the sampling process toward partitions with better scores.

Sampling Strategy To ensure that the Markov chain converges to a stable distribution, an initial burn-in phase of $10N \log N$ iterations is performed. After this phase, partitions are collected to estimate link probabilities. In practice, 1000 partitions are sampled, and to reduce correlation between consecutive samples, one partition is recorded after every N iterations.

Prediction of Links For each sampled partition, a probability score is computed for all candidate node pairs. The final score for a link is obtained by averaging over all sampled partitions.

Node pairs that are not connected in the observed network but receive high scores are considered likely missing links. On the other hand, existing edges with low scores may indicate unreliable or spurious connections.

6 Learning-Based Models

6.1 Random Forest Classifier

In contrast to similarity-based and probabilistic approaches, learning-based methods formulate link prediction as a supervised classification problem. The goal is to learn a function that predicts whether a link exists between a pair of nodes based on structural features extracted from the network.

In this work, a Random Forest classifier is used for link prediction. Random Forest is an ensemble learning technique that builds multiple decision trees and combines their predictions to improve generalization and reduce overfitting.

Feature Construction For each pair of nodes (u, v), a feature vector is constructed using structural properties of the network. The following features are used:

- **Common Neighbors (CN):**

$$CN(u, v) = |\Gamma(u) \cap \Gamma(v)| \tag{9}$$

- **Jaccard Coefficient:**

$$J(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|} \tag{10}$$

- **Preferential Attachment (PA):**

$$PA(u, v) = k_u \cdot k_v \tag{11}$$

These features capture the local structural similarity between node pairs.

Dataset Construction The dataset is constructed by considering both existing and non-existing node pairs:

- Non-edges $(u, v) \notin E$ are labeled as 0
- Existing edges $(u, v) \in E$ are labeled as 1

For each pair, the corresponding feature vector is computed, resulting in a binary classification dataset.

Model Training The dataset is split into training and testing sets using an 90:10 ratio. A Random Forest classifier is then trained on the training data.

The model parameters are defined as follows:

- Number of trees: 100
- Maximum depth: unrestricted
- Random state: 42

Each decision tree learns patterns from the input features, and the final prediction is obtained by aggregating the outputs of all trees in the ensemble.

Prediction For each node pair (u, v), the trained model outputs a probability score:

$$P(A_{uv} = 1) \tag{12}$$

which represents the likelihood of the existence of a link between the two nodes.

Summary The Random Forest-based approach leverages multiple structural features and ensemble learning to capture complex patterns in the network. By learning from both existing and non-existing edges, the model provides a flexible and data-driven framework for link prediction.

7 Experimental Setup

7.1 Datasets

The experiments are conducted on multiple real-world network datasets, commonly used for evaluating link prediction algorithms. All datasets are represented as undirected graphs.

- **Adjnoun**: A word adjacency network where nodes represent adjectives and nouns, and edges indicate adjacency in a text corpus.
- **Dolphins**: A social network of dolphins, where nodes represent dolphins and edges represent frequent associations between them.
- **Football**: A network of American college football teams, where edges represent games played between teams.
- **Karate**: Zachary's Karate Club network, representing social interactions between members.
- **Les Misérables (Lesmis)**: A co-occurrence network of characters from the novel, where edges represent joint appearances in chapters.
- **Netscience**: A collaboration network of scientists working on network theory.
- **Power**: A power grid network where nodes represent generators, transformers, and substations, and edges represent transmission lines.

7.2 Implementation Details

All algorithms are implemented using Python with the following libraries:

- **NetworkX**: Used for graph representation and structural computations.
- **NumPy**: Used for numerical operations and matrix computations.
- **Scikit-learn**: Used for evaluation metrics such as AUC, AUPR, and Average Precision.

For each dataset, edges are randomly divided into training and testing sets using a 90%-10% split. The training graph is used to compute similarity scores, and the removed edges are used as ground truth for evaluation.

8 Performance Metrics

The performance of link prediction methods is evaluated using the following metrics:

8.1 Area Under ROC Curve (AUC)

The Receiver Operating Characteristic (ROC) curve is a graphical representation of a model's performance across different classification thresholds. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR).

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \tag{13}$$

Each point on the ROC curve corresponds to a specific threshold applied to the predicted scores. By varying the threshold from 0 to 1, a series of (FPR, TPR) points are obtained, forming the ROC curve.

The Area Under the ROC Curve (AUC) summarizes this curve into a single scalar value:

$$AUC = P(s_{positive} > s_{negative}) \tag{14}$$

This can be interpreted as the probability that a randomly selected existing link is assigned a higher score than a randomly selected non-existing link.

An AUC value of 0.5 corresponds to random prediction, while a value of 1.0 indicates perfect discrimination.

8.2 Area Under Precision-Recall Curve (AUPR)

The Precision-Recall (PR) curve plots Precision against Recall for different threshold values:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN} \tag{15}$$

Each point on the PR curve represents the precision and recall achieved at a particular decision threshold. As the threshold varies, the trade-off between precision and recall is captured by the curve.

The Area Under the Precision-Recall Curve (AUPR) is defined as:

$$AUPR = \int_0^1 Precision(Recall)\, d(Recall) \tag{16}$$

A higher AUPR value indicates that the model maintains high precision across different recall levels.

Unlike AUC-ROC, AUPR is more sensitive to class imbalance. In link prediction problems, where the number of non-existing links is significantly larger than the number of existing links, AUPR provides a more reliable measure of performance.

8.3 Precision

Precision measures the proportion of correctly predicted positive links among all predicted positive links. It quantifies how many of the predicted edges are actually present in the network.

$$Precision = \frac{TP}{TP + FP} \tag{17}$$

A high precision value indicates that the model makes few false positive predictions. In the context of link prediction, this means that most of the predicted links truly exist.

8.4 Recall

Recall measures the proportion of actual links that are correctly identified by the model. It reflects the ability of the model to capture existing edges in the network.

$$Recall = \frac{TP}{TP + FN} \tag{18}$$

A high recall value indicates that the model successfully identifies most of the true links, although it may include some false positives.

8.5 Average Precision

Average Precision (AP) is a single-value summary of the Precision-Recall curve. It computes the weighted average of precision values obtained at different recall levels as the classification threshold is varied.

$$AP = \sum_k Precision(k) \cdot \Delta Recall(k) \tag{19}$$

where k denotes different threshold levels and $\Delta Recall(k)$ represents the change in recall at each step.

Average Precision can be interpreted as the area under the Precision-Recall curve, and hence is closely related to AUPR. A higher AP value indicates that the model maintains high precision across a wide range of recall values.

8.6 Recall@k

Recall@k is a ranking-based evaluation metric that measures how many true links are present among the top-k predicted links.

In link prediction, models assign a score to each candidate node pair. These pairs are then ranked in descending order of their scores. Recall@k evaluates how many actual (ground truth) links appear within the top-k ranked predictions.

$$Recall@k = \frac{\text{Number of true links in top-}k}{\text{Total number of true links}} \tag{20}$$

A higher Recall@k value indicates that the model successfully ranks true links higher among its predictions.

This metric is particularly useful in practical scenarios where only the top-ranked predictions are considered, such as recommendation systems or network completion tasks.

9 Results

9.1 AUC Comparison

Method	Adjnoun	Dolphins	Football	Karate	Lesmis	Netsci	Power
CN	0.6492	0.7475	0.9046	0.6947	0.9179	0.9339	0.5889
Jaccard	0.6720	0.9084	0.8683	0.5289	0.8688	0.9413	0.5919
PA	0.7390	0.7115	0.2385	0.7812	0.7709	0.6880	0.4507
Adamic Adar	0.6935	0.7514	0.8418	0.8672	0.9569	0.9160	0.5851
RA	0.6641	0.6912	0.8434	0.6654	0.9072	0.9452	0.5980
Cosine	0.6199	0.7625	0.8570	0.7320	0.8863	0.9250	0.5964
Sorensen	0.6376	0.7136	0.8709	0.6347	0.8697	0.9322	0.5843
CAR-PA	0.7465	0.6709	0.7434	0.5509	0.7732	0.6155	0.3679
HPI	0.6214	0.6834	0.8885	0.7196	0.8567	0.9468	0.5866
LNBCN	0.6438	0.7914	0.8454	0.5190	0.9479	0.7997	0.4986
CCLP	0.6897	0.7699	0.7726	0.7092	0.9116	0.8850	0.5505
Katz	0.8005	0.9443	0.8489	0.7622	0.8533	0.9340	0.6299
Shortest Path	0.6526	0.8415	0.7266	0.6700	0.7908	0.9333	0.6484
SimRank	0.5915	0.8331	0.9086	0.6194	0.8643	0.9419	0.6641
LHNG	0.5504	0.7546	0.8803	0.5453	0.7276	0.9437	0.6502
LP	0.7247	0.7983	0.8844	0.6906	0.8743	0.9244	0.6335
L3	0.7046	0.7374	0.8703	0.7460	0.8781	0.8805	0.5883
CH2-L2	0.6357	0.8564	0.8519	0.6951	0.9386	0.9434	0.5859
CH3-L3	0.7588	0.7292	0.8858	0.8943	0.8182	0.8877	0.5967
SBM	0.6253	0.5649	0.4909	0.6232	0.5764	0.5202	0.5643
Random Forest Classifier	0.6738	0.7152	0.8686	0.8389	0.9490	0.9649	0.7485

Table 1: AUC comparison across datasets

9.2 AUPR Comparison

Method	Adjnoun	Dolphins	Football	Karate	Lesmis	Netsci	Power
CN	0.0385	0.0955	0.2171	0.1654	0.3557	0.4771	0.0169
Jaccard	0.0125	0.0847	0.2232	0.0184	0.1637	0.4583	0.0077
PA	0.0236	0.0182	0.0061	0.0450	0.0347	0.0050	0.0001
Adamic Adar	0.0659	0.0375	0.1686	0.0892	0.4969	0.6101	0.0092
RA	0.0163	0.1018	0.2323	0.1479	0.6112	0.7083	0.0143
Cosine	0.0103	0.0416	0.2822	0.0292	0.1394	0.4488	0.0090
Sorensen	0.0111	0.0236	0.1931	0.0218	0.1442	0.4727	0.0063
CAR-PA	0.0275	0.1289	0.2656	0.0232	0.4143	0.2797	0.0024
HPI	0.0102	0.0533	0.2321	0.0917	0.2133	0.4442	0.0360
LNBCN	0.0497	0.1123	0.1897	0.1416	0.6237	0.5210	0.0161
CCLP	0.0178	0.1133	0.1350	0.0405	0.6082	0.5287	0.0119
Katz	0.0467	0.1759	0.1900	0.0460	0.3797	0.4180	0.0148
Shortest	0.3563	0.4554	0.4002	0.3905	0.4751	0.4609	0.0877
SimRank	0.0083	0.0408	0.3620	0.0199	0.0412	0.2448	0.0027
LHNG	0.0074	0.0231	0.2599	0.0166	0.0212	0.1515	0.0023
LP	0.0361	0.1239	0.2070	0.0258	0.3578	0.3978	0.0045
L3	0.0284	0.0500	0.1477	0.0512	0.4328	0.3576	0.0057
CH2-L2	0.0207	0.0734	0.2675	0.1465	0.5926	0.6881	0.0185
CH3-L3	0.0717	0.0288	0.1059	0.3231	0.2023	0.2402	0.0060
SBM	0.0145	0.0104	0.0096	0.0281	0.0326	0.0002	0.0030
Random Forest Classifier	0.1735	0.2311	0.6482	0.5857	0.7673	0.8487	0.0706

Table 2: AUPR comparison across datasets

9.3 Average Precision Comparison

Method	Adjnoun	Dolphins	Football	Karate	Lesmis	Netsci	Power
CN	0.0363	0.0583	0.2086	0.1012	0.3248	0.4049	0.0085
Jaccard	0.0121	0.0918	0.2217	0.0224	0.1487	0.3952	0.0017
PA	0.0256	0.0198	0.0068	0.0593	0.0382	0.0054	0.0001
Adamic Adar	0.0760	0.0406	0.1735	0.1185	0.5057	0.6017	0.0056
RA	0.0172	0.0998	0.2355	0.1502	0.6146	0.6929	0.0084
Cosine	0.0099	0.0436	0.2891	0.0360	0.1280	0.3933	0.0021
Sorensen	0.0108	0.0236	0.1992	0.0248	0.1323	0.4116	0.0013
CAR-PA	0.0325	0.1354	0.2753	0.0322	0.4232	0.2797	0.0024
HPI	0.0097	0.0206	0.2364	0.0357	0.0475	0.1731	0.0017
LNBCN	0.0514	0.1140	0.1980	0.1412	0.6249	0.5173	0.0115
CCLP	0.0183	0.1154	0.1422	0.0573	0.6090	0.5230	0.0072
Katz	0.0508	0.1865	0.1985	0.0639	0.3857	0.4183	0.0151
Shortest	0.0115	0.0325	0.0209	0.0246	0.0256	0.0591	0.0025
SimRank	0.0086	0.0452	0.3689	0.0231	0.0435	0.2365	0.0027
LHNG	0.0077	0.0259	0.2712	0.0210	0.0228	0.1598	0.0023
LP	0.0403	0.1418	0.2136	0.0309	0.3597	0.3953	0.0027
L3	0.0324	0.0622	0.1526	0.0665	0.4403	0.3556	0.0045
CH2-L2	0.0268	0.0917	0.2740	0.1816	0.5930	0.6817	0.0138
CH3-L3	0.0758	0.0379	0.1111	0.3476	0.2240	0.2386	0.0045
SBM	0.0158	0.0115	0.01	0.0354	0.0147	0.0002	0.0029
Random Forest Classifier	0.1854	0.2617	0.6456	0.5669	0.7602	0.8387	0.0699

Table 3: Average Precision comparison across datasets

9.4 Recall@K Comparison

Method	Adjnoun	Dolphins	Football	Karate	Lesmis	Netsci	Power
CN	0.0465	0.0625	0.3226	0.1250	0.3846	0.4364	0.0409
Jaccard	0.0000	0.0625	0.3065	0.0000	0.1923	0.5273	0.0106
PA	0.0465	0.0000	0.0000	0.0000	0.0769	0.0400	0.0000
Adamic Adar	0.1628	0.0625	0.2903	0.1250	0.5000	0.5964	0.0303
RA	0.0465	0.1250	0.3710	0.1250	0.5769	0.6545	0.0273
Cosine	0.0000	0.0625	0.3871	0.0000	0.0385	0.5055	0.0091
Sorensen	0.0000	0.0000	0.2903	0.0000	0.1154	0.5018	0.0106
CAR-PA	0.0698	0.2500	0.3710	0.0000	0.5000	0.2909	0.0106
HPI	0.0000	0.0000	0.3226	0.0000	0.0385	0.1673	0.0167
LNBCN	0.0698	0.0625	0.2581	0.1250	0.5385	0.5200	0.0409
CCLP	0.0233	0.1250	0.2258	0.1250	0.6154	0.5091	0.0333
Katz	0.1395	0.2500	0.3065	0.1250	0.3462	0.4691	0.0530
Shortest	0.0233	0.0625	0.0161	0.0000	0.0385	0.0545	0.0045
SimRank	0.0000	0.0000	0.3710	0.0000	0.0000	0.3055	0.0061
LHNG	0.0000	0.0000	0.3065	0.0000	0.0000	0.1709	0.0000
LP	0.0930	0.2500	0.2581	0.0000	0.3077	0.3964	0.0227
L3	0.0930	0.0625	0.1290	0.0000	0.4615	0.3600	0.0288
CH2-L2	0.0465	0.1250	0.4194	0.2500	0.5769	0.6073	0.0303
CH3-L3	0.1395	0.0625	0.1290	0.3750	0.2692	0.2582	0.0424
SBM	0.0233	0.0000	0.0000	0.0000	0.0385	0.0000	0.0223
Random Forest Classifier	0.225	0.4	0.7454	0.6363	0.7272	0.7846	0.1566

Table 4: Recall@K comparison across datasets

10 Comparison of Methods

In this section, we compare the performance of different link prediction methods based on the experimental results obtained across multiple datasets.

10.1 Local vs Global Methods

Local similarity-based methods such as Common Neighbors (CN), Resource Allocation (RA), and Jaccard rely only on neighborhood information. These methods consistently achieve strong performance across most datasets, particularly in terms of AUC.

In contrast, global methods such as Katz and shortest-path-based approaches utilize information from the entire network. While these methods can capture long-range dependencies, their performance gains are not consistently significant compared to local methods. Additionally, they incur higher computational cost, making them less scalable.

10.2 Community-Aware Methods

Methods that incorporate local community structure, such as CAR, CH2-L2, and CH2-L3, outperform traditional similarity indices across all evaluation metrics. These methods effectively exploit the tendency of nodes within the same local region to form links, leading to superior performance in AUC, AUPR, and Recall@k.

This observation highlights the importance of community structure in real-world networks.

10.3 Probabilistic vs Heuristic Methods

The stochastic block model (SBM), a probabilistic approach, provides a different perspective by modeling link formation as a generative process. While SBM achieves stable and consistent results, it does not outperform the best heuristic methods in terms of accuracy.

However, SBM is particularly useful in scenarios involving noisy or incomplete networks, as it explicitly models uncertainty in link existence.

10.4 Learning-Based Methods

Learning-based approaches, such as the Random Forest classifier, leverage multiple structural features to learn complex patterns in the network. Unlike individual similarity indices, these models can capture non-linear relationships and interactions between different topological features.

From the experimental results, the Random Forest classifier consistently achieves higher performance across most datasets, particularly in terms of AUC, AUPR, and Recall@K. In datasets such as Netscience, Lesmis, and Football, it outperforms both heuristic and probabilistic methods by a significant margin. This demonstrates the advantage of combining multiple features over relying on a single similarity measure.

However, their performance depends heavily on feature quality and dataset characteristics. Additionally, they require labeled data and training, which may not always be available. There is also a potential risk of overfitting, especially in smaller datasets, which should be considered when interpreting the results.

10.5 Effect of Evaluation Metrics

A comparison across evaluation metrics reveals that:

- AUC values are generally high for most methods, indicating strong ranking ability across the entire set of node pairs.
- AUPR values are significantly lower, reflecting the challenge of class imbalance in link prediction, where non-existing links dominate.
- Average Precision (AP) follows trends similar to AUPR, as both focus on the quality of ranking for positive links. Methods such as Resource Allocation, CCLP, and CH2-L2 achieve high AP values, indicating their effectiveness in identifying true links among top-ranked predictions. Learning-based methods, particularly Random Forest, achieve the highest AP values across most datasets, demonstrating superior precision-recall trade-offs.
- Recall@K highlights the effectiveness of top-ranked predictions. Community-aware methods and Random Forest perform particularly well in this metric, indicating their ability to correctly identify a large fraction of missing links within the top-K predictions.

11 Discussion

The experimental results reveal several important insights regarding the performance of different link prediction methods across diverse network datasets.

- **Learning-based methods achieve the best overall performance:** The Random Forest classifier consistently achieves the highest performance across most datasets in terms of AUC, AUPR, Average Precision, and Recall@K. Compared to individual similarity indices, it significantly improves prediction accuracy by combining multiple structural features and capturing complex, non-linear relationships. This highlights a clear performance gap between learning-based and heuristic approaches.
- **Community-aware methods are the strongest heuristic approaches:** Advanced local methods such as CAR, CH2-L2, and CH3-L3 consistently outperform traditional similarity indices like CN and Jaccard. In particular, CH2-L2 and CH3-L3 achieve higher AUPR and Recall@K values, indicating better identification of true links among top-ranked predictions. This demonstrates the advantage of incorporating local community structure over simple neighbor counting.
- **Local similarity methods provide a strong baseline:** Methods such as Common Neighbors (CN) and Resource Allocation (RA) achieve competitive performance across multiple datasets. While they are generally outperformed by community-aware and learning-based methods, they offer a good trade-off between accuracy and computational efficiency. Resource Allocation, in particular, often performs better than CN due to its penalization of high-degree nodes.

- **Preferential Attachment performs poorly:** The PA index shows consistently low performance compared to other local methods such as CN and RA. This confirms that relying solely on node degree is insufficient for accurate link prediction in most real-world networks.

- **Global methods offer limited advantage over quasi-local methods:** Although global methods such as Katz and SimRank capture long-range dependencies, their performance is often comparable to or slightly better than local methods, but generally inferior to quasi-local methods like L3 and CH2-L2. Given their higher computational cost, this limits their practical usefulness in large-scale networks.

- **Probabilistic methods show stable but moderate performance:** The stochastic block model (SBM) produces consistent results but does not match the performance of top heuristic or learning-based methods. Compared to community-aware heuristics, SBM is less effective in capturing fine-grained local structure, although it remains useful in noisy or incomplete networks.

- **Impact of class imbalance on evaluation metrics:** AUPR and Average Precision values are significantly lower compared to AUC across all methods. While AUC indicates good overall ranking ability, AUPR and AP reveal that distinguishing true links among top predictions is more challenging. Methods such as CH2-L2, CCLP, and Random Forest perform better under these stricter metrics.

- **Trade-off between accuracy and complexity:** Random Forest provides the highest accuracy but requires feature engineering, labeled data, and additional computational resources. In contrast, heuristic methods are simpler, faster, and more interpretable. Among heuristics, community-aware methods offer the best balance, while basic local methods remain suitable for large-scale scenarios.

- **Dependence on network structure:** The relative performance of methods varies across datasets. Dense and highly clustered networks favor local and community-aware methods, while sparse networks reduce the effectiveness of purely local approaches. Learning-based methods remain more robust across varying structural conditions.

12 Conclusion

In this project, we implemented and evaluated a wide range of link prediction techniques, including similarity-based methods, probabilistic models, and learning-based approaches, on multiple real-world network datasets.

The experimental results demonstrate that simple local similarity-based methods remain highly effective for link prediction, particularly due to their ability to capture local structural patterns such as common neighbors and clustering effects. Advanced variants such as CAR-based and CH2-L2/CH3-L3 indices further improve performance by incorporating local community information, which is a key factor in real-world network formation.

Learning-based approaches, particularly the Random Forest classifier, achieve the best overall performance across most datasets. By combining multiple structural features, these models are able to capture complex patterns that are not exploited by individual

similarity indices. However, this improved performance comes at the cost of increased computational complexity and the need for labeled data and feature engineering.

Probabilistic approaches, such as the stochastic block model, provide a principled framework for modeling link formation and handling noisy data. However, their computational complexity and parameter estimation challenges limit their applicability to large-scale networks.

The study also highlights the importance of evaluation metrics. While AUC provides a general measure of ranking performance, AUPR and Average Precision offer a more realistic evaluation in highly imbalanced scenarios, which is typical in link prediction problems.

Overall, the results confirm that no single heuristic method performs best across all datasets, as also discussed in the survey paper. The effectiveness of a method depends on the structural characteristics of the network, such as sparsity, clustering, and degree distribution. Therefore, selecting an appropriate method requires careful consideration of both network properties and computational constraints.

This work emphasizes that combining structural insights with advanced modeling techniques, including learning-based approaches, leads to improved link prediction performance and provides a foundation for further research in more complex and dynamic network settings.

Submitted by

Ojus Krishna
Roll No. 24075099